BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name:

ID:

> **Due Date**
>
> Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `~/BEE_4750/hw4-evelyn-natalie`
```

```julia
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost ($/ha-yr)	Selling Price ($/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

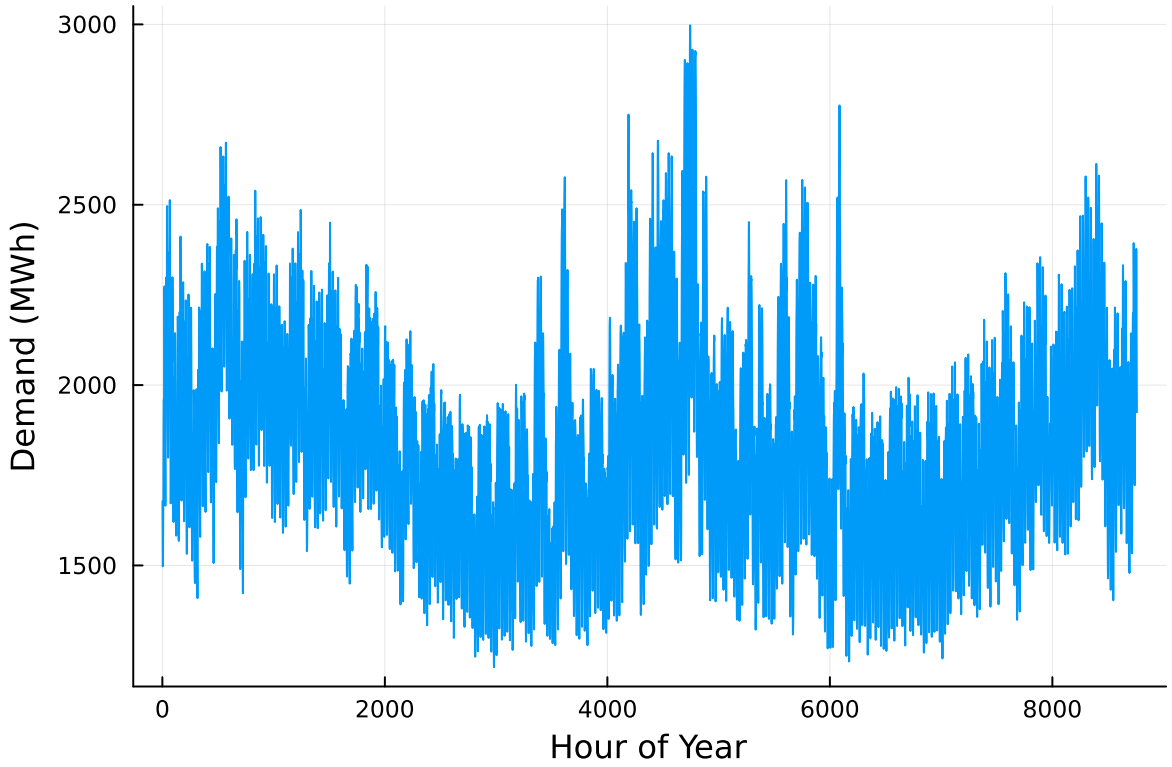
Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

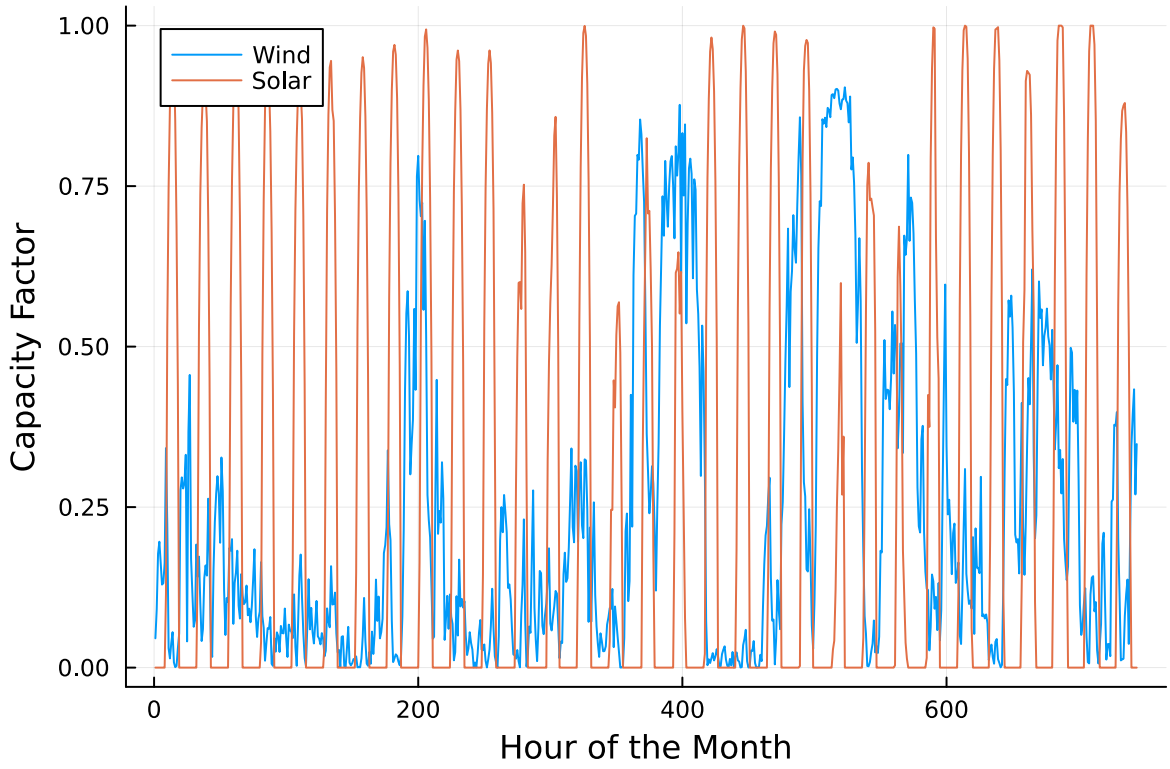
	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

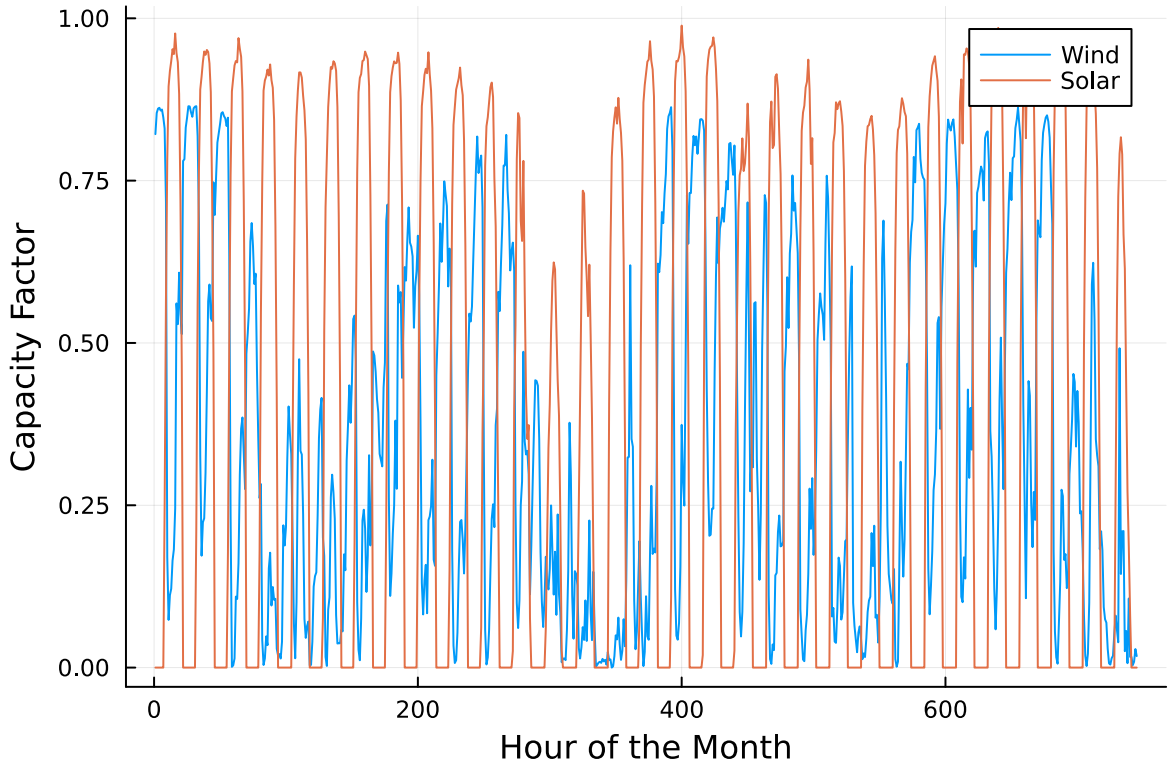
Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing

output by adding a semi-colon after the last command, or you might find that your notebook crashes.

Problem 3 Solutions

Problem 3.1

Our decision variables:

C_i = Installed capacity of generator type i (MW)

$G_{i,t}$ = Electricity generated by plant type i at hour t (MWh)

NSE_t = Non-served demand at hour t (MWh)

Our objective functions: We want to minimize the cost of the system.

The cost of our system consists of the investment cost + operating costs = $\sum$ (Fixed Costs) ($installed\ capacity\ from\ each\ generator$) + $\sum\sum$ (Variable Costs) ($production\ from\ each\ generator$) + $\sum\ NSE_{cost}\ NSE_t$. It is given that the utility will penalize any non-served demand at a rate of \$10,000/MWh, which we cna denote using P_{NSE}.

Thus,

$$minZ = \sum(F_i)(C_i) + \sum\sum(V_i)(G_{i,t}) + \sum(P_{NSE})(NSE_t)$$

Our constraints:

Demand balance: Electricity demand at hour t is equal to non-served demand at hour t + electrcity generated by plant type at hour t$$

$$\sum(G_{i,t}) + NSE_t = D_t$$

CO_2 constraint: The NY state legislature is planning to enact an annual CO2 limit ($CO2_{max}$), which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

$$\sum\sum E_i * G_{i,t} \leq CO2_{max}$$

Capacity constraint: While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology.

For coal, CCGT, and CT:

$$G_{i,t} \leq C_i$$

For geothermal:

$$G_{i,t} \leq 0.85 C_i$$

For wind and solar: we need to account for capacity factor of generator i at hour t.

$$G_{i,t} \leq CF_{i,t} * C_i$$

Non-negativity constraint: decision variables cannot be negative.

$$C_i, G_{i,t}, NSE_t \geq 0,$$

```
# load files and reformat dates
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)
T = nrow(demand)
hours = 1:T

generators = DataFrame(CSV.File("data/generators.csv"))
plants = collect(gens.Plant)

capacity_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# retrieve parameters from csv
fixed_costs = Dict{String,Float64}()
variable_costs = Dict{String,Float64}()
emissions = Dict{String,Float64}()
for i in eachrow(generators)
    pname = String(i.Plant)
    fixed_costs[pname] = float(i.FixedCost)
    variable_costs[pname] = float(i.VarCost)
    emissions[pname] = float(i.Emissions)
end

# build dictionary of capacity factors for wind, solar, geothermal (85%), and coal/ccgt/ct (
CF = Dict{String,Vector{Float64}}()
for i in plants
    if i == "Wind"
        CF[i] = cap_factor.Wind[1:T]
    elseif i == "Solar"
        CF[i] = cap_factor.Solar[1:T]
```

```
    elseif i == "Geothermal"
        CF[i] = fill(0.85, T)
    else
        CF[i] = ones(T)
    end
end

# Other parameters
NSE = 10_000.0 # $/MWh
CO2_max = 1.5e6 # tCO2/yr

# convert demand to Float64 (MWh)
Demand = [float(demand.Demand[t]) for t in hours]

# build model
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, Capacity_installed[i in plants] >= 0) # MW
@variable(model, Generation[i in plants, t in hours] >= 0) # MWh in hour t
@variable(model, Nonserved[t in hours] >= 0) # MWh

# demand constraint
demand_constraint = Vector{ConstraintRef}(undef, T)

@objective(model, Min,
    sum(fixed_costs[i]*Capacity_installed[i] for i in plants) +
    sum(variable_costs[i]*Generation[i,t] for i in plants, t in hours) +
    sum(NSE*Nonserved[t] for t in hours)
)

# demand balance constraints
for t in hours
    demand_constraint[t] = @constraint(model, sum(Generation[i,t] for i in plants) + Nonserve
end

# capacity constraints
capacity_constraint = Dict{Tuple{String,Int}, ConstraintRef}()
for i in plants
    for t in hours
        capacity_constraint[i,t] = @constraint(model, Generation[i,t] <= CF[i][t] * Capacity_
    end
```

```
end
# CO2 constraint
co2_constraint = @constraint(model, sum(emissions[i]*Generation[i,t] for i in plants, t in ho

optimize!(model)

total_cost = objective_value(model)

NSE_vals = [value(Nonserved[t]) for t in hours]
total_nonserved_MWh = sum(NSE_vals)

# annual generation for each plant
annual_gen = Dict(i => sum(value.(Generation[i,t]) for t in hours) for i in plants)
total_generation = sum(values(annual_gen))

# fraction of generation versus capacity
fraction_generation = Dict(i => (annual_gen[i] / total_generation) for i in plants)
fraction_capacity = Dict(i => (Capacity_vals[i] / sum(values(Capacity_vals))) for i in plants

# hourly prices: duals of demand constraints
hourly_prices = [dual(demand_constraint[t]) for t in hours]  # $/MWh

# CO2 dual (shadow price $/tCO2)
co2_shadow = dual(co2_constraint)
value_1000t = co2_shadow * 1000.0

# display results
println("-----------")
println("Problem 3.2")
println("-----------")
for i in plants
    println("Installed capacity for ", i, " : ", round(Capacity_vals[i]; digits=2), " MW, cap
end
println("The total cost is \$", round(total_cost, digits=2))
println("Total non-served energy (MWh/yr): ", round(total_nonserved_MWh; digits=2))

println("-----------")
println("Problem 3.3")
println("-----------")
for i in plants
    println(i, " generation: ", round(annual_gen[i]; digits=1),
            " MWh (", round(100*fraction_generation[i]; digits=2), "% of generation)")
```

```
end

println("-----------")
println("Problem 3.5")
println("-----------")
println("CO2 shadow price: \$", round(co2_shadow; digits=4), " per tCO2")
println("The value of allowing an additional 1000 tCO2/yr is \$", round(value_1000t; digits=
```

```
-----------
Problem 3.2
-----------
Installed capacity for Geothermal : 285.57 MW, capacity % = 3.99%
Installed capacity for Coal : 0.0 MW, capacity % = 0.0%
Installed capacity for NG CCGT : 1509.17 MW, capacity % = 21.11%
Installed capacity for NG CT : 703.06 MW, capacity % = 9.83%
Installed capacity for Wind : 2548.89 MW, capacity % = 35.65%
Installed capacity for Solar : 2103.75 MW, capacity % = 29.42%
The total cost is $7.8535295894e8
Total non-served energy (MWh/yr): 332.29
-----------
Problem 3.3
-----------
Geothermal generation: 1.0969955e6 MWh (6.7% of generation)
Coal generation: 0.0 MWh (0.0% of generation)
NG CCGT generation: 3.3227666e6 MWh (20.3% of generation)
NG CT generation: 129473.4 MWh (0.79% of generation)
Wind generation: 6.4513419e6 MWh (39.42% of generation)
Solar generation: 5.3669963e6 MWh (32.79% of generation)
-----------
Problem 3.5
-----------
CO2 shadow price: $-182.7719 per tCO2
The value of allowing an additional 1000 tCO2/yr is $-182771.94
```

Problem 3.2

The utility should build the following installed capacities of each type of generating plant: 285.57 MW of geothermal, 0.0 MW of coal, 1509.17 MW of NG CCGT, 703.06 MQ of NG CT, 2548.89 MW of wind, and 2103.75 ME of solar.

The total cost per year would be the combination of the total fixed costs per year = $7.8535295894e8.

The total non-served energy is 332.29 MWh/yr.

Problem 3.3

The frcation of annual generation that each plant type produces is below: Geothermal should generate 1.0969955e6 MWh (6.7% of generation, capacity share = 3.99%).

Coal doesn't generate anything (0% of generation and capacity share).

NG CCGT generates 3.3227666e6 MWh (20.3% of generation, capacity share = 21.11%).

NG CT generates 129473.4 MWh (0.79% of generation, capacity share = 9.83%).

Wind generates 6.4513419e6 MWh (39.42% of generation, capacity share = 35.65%).

Solar generates 5.3669963e6 MWh (32.79% of generation, capacity share = 29.42%)

We can see here that the largest percentage of generation comes from solar and wind energy (33% and 39% respectively). For both solar and wind, the % of installed capacity is slightly lower than the % of generation (29% and 36% respectively), which makes sense because these variable renewables are cheaper to run (due to no fuel cost). NG CCGT generates 20%, with % of capacity 21%. NG CT generates a tiny percentage (1%), with % of capacity 10%. This is because NG CT is only used for a few hours when demand is high. Coal is expensive because of the fuel costs, so it is not used at all. Geothernal is also used a little.

Problem 3.5

The shadow price of CO2 is -182.7719 dollars per tCO2. Because the shadow price is negative, this means the total cost would actually increase if the CO2 limit was relaxed. Allowing more emissions (lke adding 1000 tCO2) would actually increase the objective cost. The value of allowing an additional emission 1000 tCO2/yr would actually cause the utility to lose -182771.94 dollars a year.

References

List any external references consulted, including classmates.

Thank you to TA Samantha for providing much-needed assistance on Problem 3—explaining the linear program and walking me through the decision variables, objective functions, other parameters, and the constraints. Professor Srikrishnan's slides were also consulted, specifically "Shadow Prices and Capacity Expansion" as well as "Lab: Linear Programming", to aid my understanding of JUMP analysis + optimization. For the coding aspect, I consulted Jump.dev docuementation and Julia packages documentation to help me structure my code and understand the code function formats. Lastly, I had trouble retrieving parameters (plant

name, fixed costs, variable costs, and emissions) from the generators CSV file., as I ran into debugging errors. I consulted ChatGPT, which provided me with the correct Julia syntax.